Fejlett képfeldolgozási módszerek

5. beadandó feladat

Feladat leírás

A feladat célja egy deep learning alapú képklasszifikációs pipeline felépítése PyTorch környezetben. A megoldás során egy előre tanított (pretrained) modellt kell adaptálni egy specifikus adathalmazra (fine-tuning). Továbbá implementálni kell a teljes tanítási, validációs és kiértékelési folyamatot is.

Részfeladatok

1. **Adat-előkészítés és augmentáció**

   - Töltsük be a választott adathalmazt (MNIST, FashionMNIST, CIFAR10, stb.) a `torchvision.datasets` segítségével.
   - Bontsuk fel tanító, validáló, tesztelő alhalmazokra.
   - A tanító adathalmazon alkalmazzunk legalább háromféle online augmentációt (pl. RandomHorizontalFlip, RandomRotation, ColorJitter) a robusztusság növelése érdekében.
   - Minden képet méretezzünk át a modellnek megfelelő méretre (pl.: EfficientNet input képmérete 224×224), és alkalmazzuk a standard ImageNet normalizációt (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
   - Hozzunk létre DataLoader-eket a tanító, validációs és teszt halmazokhoz.
     - Figyeljünk rá, hogy a validációs és teszt DataLoader esetén NE alkalmazzunk augmentációt, csak átméretezést és normalizációt!

2. **Transfer learning és modellépítés**

   - Töltsünk be egy előre tanított modellt a `torchvision.models` csomagból (pl. ResNet-18 vagy EfficientNet-B0, de lehet más is).
   - Módosítsuk a hálózat utolsó rétegét (classifier/fc), hogy a kimeneti neuronok száma megegyezzen a saját adathalmazunk osztályainak számával.
   - Állítsuk a modellt fine-tuning módra: minden réteg súlya legyen frissíthető.
   - **Megjegyzés**: Nem mindegyik modell képes by default grayscale képeket bemenetként kezelni (pl.: EfficientNet RGB képet vár bemenetként)!

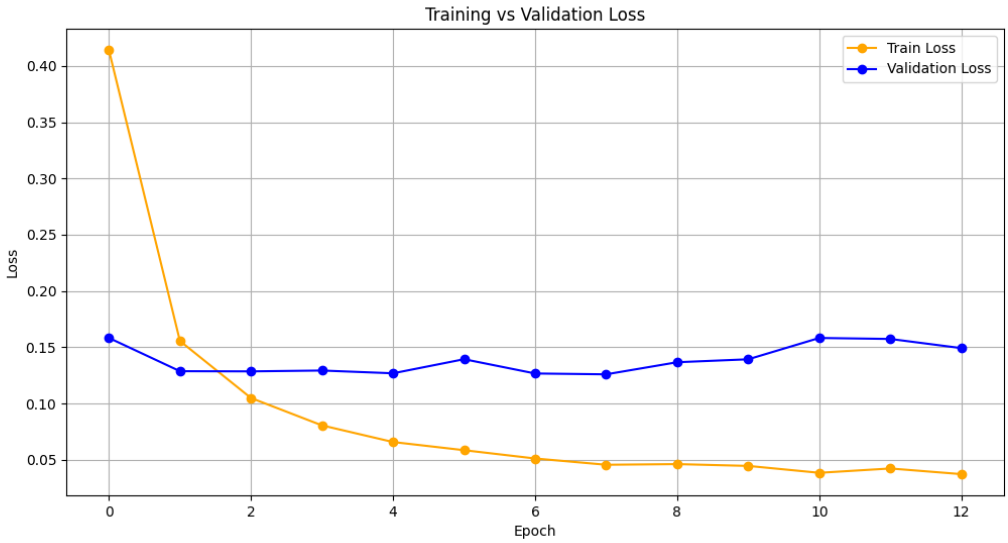
3. **A tanítási folyamat implementálása**

   - Válasszunk megfelelő veszteségfüggvényt (pl. CrossEntropyLoss) és optimalizálót (pl. Adam vagy SGD).
   - Definiáljuk a hiperparamétereket: epoch szám, batch méret, learning rate, stb.

- Implementáljuk a tanítási ciklust (training loop), amely minden batch-re elvégzi a forward passt, a hiba számítását, a visszaterjesztést (backpropagation) és a súlyfrissítést.
- Készítsünk validációs ciklust, amely minden epoch végén méri a modell teljesítményét a validációs halmazon `model.eval()` és `torch.no_grad()` alkalmazásával.
- Alkalmazzunk early stoppingot is, csak a legjobb modellt mentsük el.

4. **Kiértékelés és Vizualizáció**

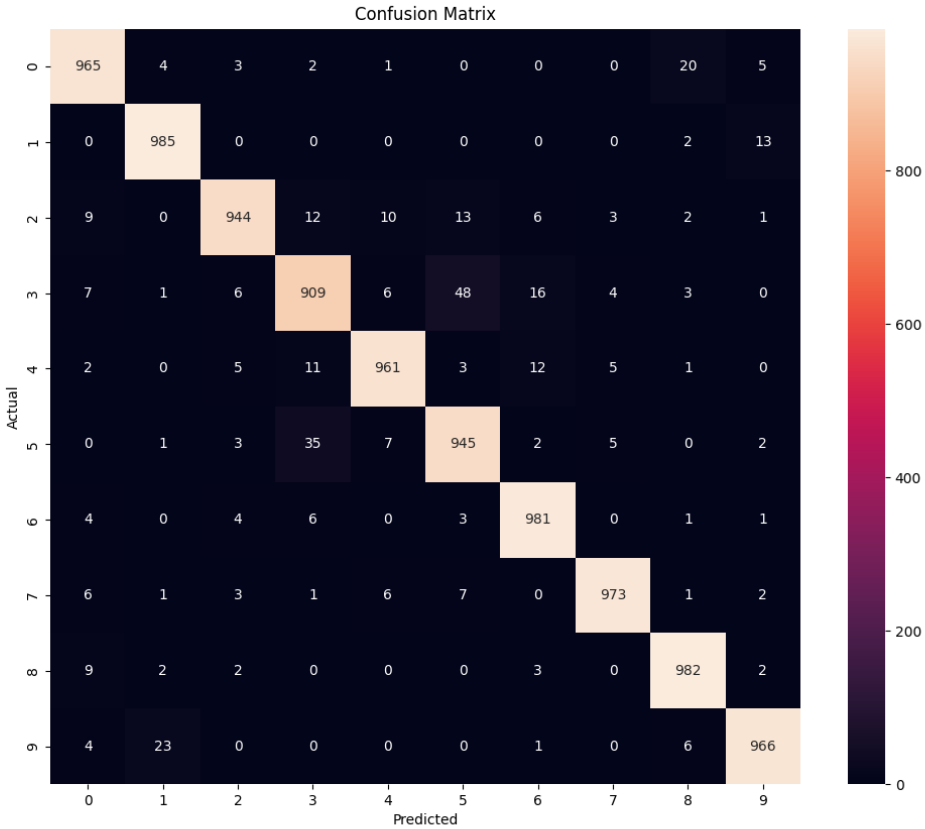
   - A tanítás befejeztével értékeljük ki a modellt a teszt halmazon.
   - Számítsuk ki az összesített pontosságot (accuracy), és készítsünk egy keveredési mátrixot (confusion matrix) a hibák , lásd 2. ábra.
   - Ábrázoljuk grafikonon a tanítási és validációs veszteség (loss), valamint a pontosság alakulását az epoch-ok függvényében, lásd 1. ábra.



1. ábra. A két loss függvény (train és valid) megjelenítve.

Formai követelmények

- Generatív AI használata ***nem megengedett***!
- Ajánlott, használható csomagok:
  - PyTorch
  - TorchVision
  - Matplotlib
  - Seaborn
  - tqdm
  - Scikit-Learn



2. ábra. Keveredési mátrix a teszt halmazon.

- A megoldásokat egy **.zip fájlban** - NEPTUNKÓD.zip formában - kérjük feltölteni a tárgy Moodle oldalára. A .zip fájl tartalmazza:
  - Python scriptet (.py vagy .ipynb)
  - Kimeneti képeket (.jpg vagy .png)
  - Minimum 1, maximum 2 oldalas dokumentumot (.pdf vagy .docx) a megoldás és az eredmények dokumentálásáról.
- A végleges kód ne egyetlen monolit blokk legyen: a részfeladatokat (adathalmaz, modell betöltés, train és valid loop, stb.) különálló függvényekben valósuljanak meg.
- A feladat beküldési határideje: **2025.05.01. - 16:00**